

Trần Tấn Phát
MSSV: 2274802010644

LAB8

Bài làm

Cho Life Style Data

1. Xử lý dữ liệu cơ bản bằng python(Nếu có) Gợi ý: Kiểm tra dữ liệu thiếu
2. Vẽ các biểu đồ để khám phá dữ liệu
3. Xây dựng một dashboard đơn giản lấy cảm hứng từ một chủ đề có trong tập data do các bạn tự chọn
4. Sử dụng mô hình hồi quy tuyến tính để dự đoán chỉ số BMI
5. Sử dụng flask để trực quan hóa Dashboard và mô hình học máy đã sử dụng lên web

1. Xử lý và kiểm tra dữ liệu thiếu

```
2. """
3. Script xử lý dữ liệu cơ bản - Kiểm tra dữ liệu thiếu
4. """
5. import pandas as pd
6. import numpy as np
7.
8. def load_data(file_path):
9.     """Load dữ liệu từ file CSV"""
10.    try:
11.        df = pd.read_csv(file_path)
12.        print(f"✓ Đã tải dữ liệu từ {file_path}")
13.        print(f" Số dòng: {len(df)}, Số cột: {len(df.columns)}")
14.        return df
15.    except Exception as e:
16.        print(f"X Lỗi khi tải dữ liệu: {e}")
17.        return None
18.
19. def check_missing_data(df):
20.     """Kiểm tra dữ liệu thiếu"""
21.     print("\n" + "="*60)
22.     print("KIỂM TRA DỮ LIỆU THIẾU")
23.     print("="*60)
24.
25.     # Đếm số giá trị thiếu
26.     missing_count = df.isnull().sum()
```

```

27. missing_percent = (missing_count / len(df)) * 100
28.
29. # Tạo bảng tổng hợp
30. missing_df = pd.DataFrame({
31.     'Cột': missing_count.index,
32.     'Số giá trị thiếu': missing_count.values,
33.     'Phần trăm thiếu (%)': missing_percent.values
34. })
35.
36. # Lọc các cột có dữ liệu thiếu
37. missing_df = missing_df[missing_df['Số giá trị thiếu'] > 0].sort_values('Số giá trị thiếu', ascending=False)
38.
39. if len(missing_df) == 0:
40.     print("\n✓ Không có dữ liệu thiếu!")
41. else:
42.     print(f"\n⚠ Có {len(missing_df)} cột có dữ liệu thiếu:\n")
43.     print(missing_df.to_string(index=False))
44.
45. return missing_df
46.
47. def get_data_info(df):
48.     """Hiển thị thông tin tổng quan về dữ liệu"""
49.     print("\n" + "="*60)
50.     print("THÔNG TIN TỔNG QUAN VỀ DỮ LIỆU")
51.     print("="*60)
52.
53.     print(f"\nKích thước dữ liệu: {df.shape[0]} dòng x {df.shape[1]} cột")
54.     print(f"\nKiểu dữ liệu của các cột:")
55.     print(df.dtypes)
56.
57.     print(f"\nThống kê mô tả (số):")
58.     numeric_cols = df.select_dtypes(include=[np.number]).columns
59.     print(df[numeric_cols].describe())
60.
61.     print(f"\nThống kê mô tả (categorical):")
62.     categorical_cols = df.select_dtypes(include=['object']).columns
63.     if len(categorical_cols) > 0:

```

```

64.     for col in categorical_cols[:10]: # Giới hạn 10 cột đầu
65.         print(f"\n{col}:")
66.         print(df[col].value_counts().head())
67.
68. def clean_data(df):
69.     """Làm sạch dữ liệu cơ bản"""
70.     print("\n" + "="*60)
71.     print("LÀM SẠCH DỮ LIỆU")
72.     print("="*60)
73.
74.     initial_shape = df.shape
75.
76.     # Loại bỏ các cột không cần thiết hoặc có quá nhiều giá trị thiếu (>50%)
77.     cols_to_drop = []
78.     for col in df.columns:
79.         missing_pct = (df[col].isnull().sum() / len(df)) * 100
80.         if missing_pct > 50:
81.             cols_to_drop.append(col)
82.
83.     if cols_to_drop:
84.         print(f"\nLoại bỏ {len(cols_to_drop)} cột có >50% dữ liệu thiếu:")
85.         for col in cols_to_drop:
86.             print(f" - {col}")
87.         df = df.drop(columns=cols_to_drop)
88.
89.     # Loại bỏ các dòng trùng lặp
90.     duplicates = df.duplicated().sum()
91.     if duplicates > 0:
92.         print(f"\nLoại bỏ {duplicates} dòng trùng lặp")
93.         df = df.drop_duplicates()
94.
95.     # Điền giá trị thiếu cho các cột số bằng giá trị trung bình
96.     numeric_cols = df.select_dtypes(include=[np.number]).columns
97.     for col in numeric_cols:
98.         if df[col].isnull().sum() > 0:
99.             mean_val = df[col].mean()
100.            df[col].fillna(mean_val, inplace=True)

```

```

101.     print(f"\nĐiền giá trị thiếu cho {col} bằng giá trị trung bình: {mean_val:.2f}")
102.
103. # Điền giá trị thiếu cho các cột categorical bằng mode
104. categorical_cols = df.select_dtypes(include=['object']).columns
105. for col in categorical_cols:
106.     if df[col].isnull().sum() > 0:
107.         mode_val = df[col].mode()[0] if len(df[col].mode()) > 0 else 'Unknown'
108.         df[col].fillna(mode_val, inplace=True)
109.     print(f"\nĐiền giá trị thiếu cho {col} bằng mode: {mode_val}")
110.
111. final_shape = df.shape
112. print(f"\nKích thước trước khi làm sạch: {initial_shape}")
113. print(f"\nKích thước sau khi làm sạch: {final_shape}")
114.
115. return df
116.
117. def main():
118.     """Hàm chính"""
119.     print("="*60)
120.     print("XỬ LÝ DỮ LIỆU CƠ BẢN")
121.     print("="*60)
122.
123.     # Load dữ liệu
124.     df_final = load_data('data/Final_data.csv')
125.     df_meal = load_data('data/meal_metadata.csv')
126.
127.     if df_final is not None:
128.         print("\n" + "-"*60)
129.         print("XỬ LÝ FILE Final_data.csv")
130.         print("-"*60)
131.         check_missing_data(df_final)
132.         get_data_info(df_final)
133.         df_final_cleaned = clean_data(df_final)
134.         df_final_cleaned.to_csv('data/Final_data_cleaned.csv', index=False)
135.         print("\n✓ Đã lưu file đã làm sạch: data/Final_data_cleaned.csv")
136.
137.     if df_meal is not None:

```

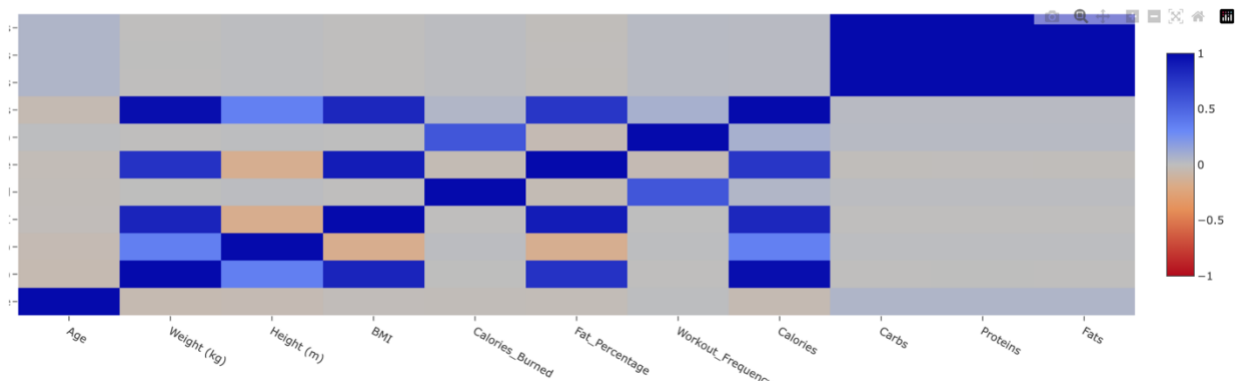
```

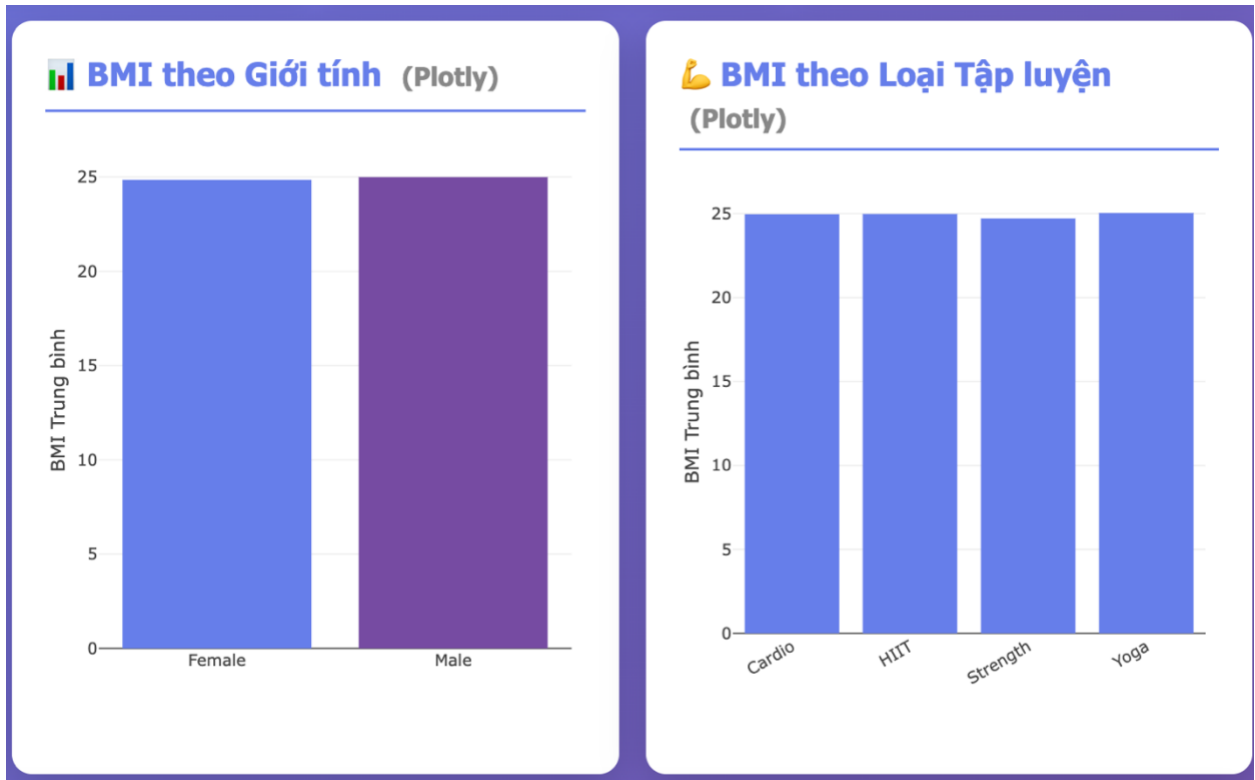
138. print("\n" + "-"*60)
139. print("XỬ LÝ FILE meal_metadata.csv")
140. print("-"*60)
141. check_missing_data(df_meal)
142. get_data_info(df_meal)
143. df_meal_cleaned = clean_data(df_meal)
144. df_meal_cleaned.to_csv('data/meal_metadata_cleaned.csv', index=False)
145. print("\n✓ Đã lưu file đã làm sạch: data/meal_metadata_cleaned.csv")
146.
147. if __name__ == "__main__":
148.     main()
149.

```

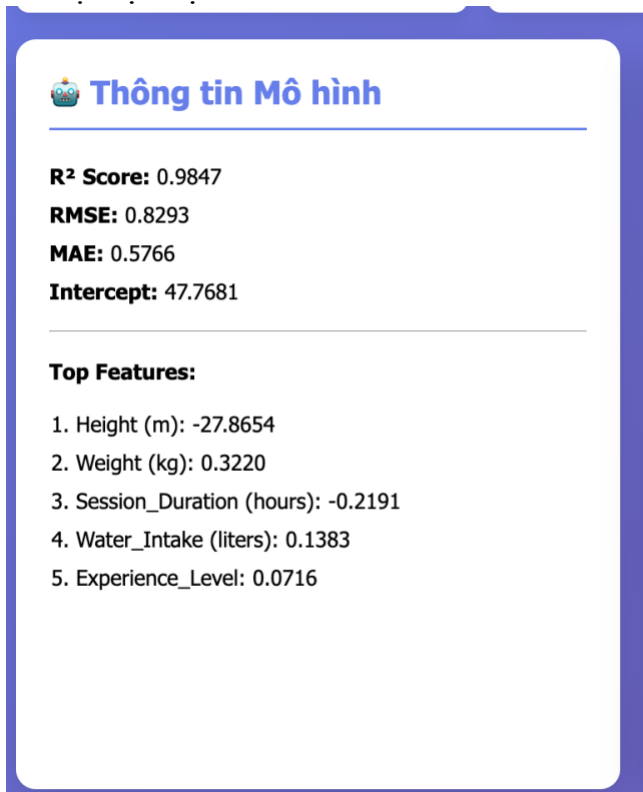
2. Vẽ biểu đồ khám phá dữ liệu

🔥 **Bản đồ Tương quan** (Plotly)





3. Xây dựng một dashboard đơn giản lấy cảm hứng từ một chủ đề có trong tập data do các bạn tự chọn



4. Sử dụng mô hình hồi quy tuyến tính để dự đoán chỉ số BMI

"""

Xây dựng mô hình hồi quy tuyến tính để dự đoán BMI

"""

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

```
import pickle
```

```
import os
```

```
def prepare_data(df):
```

```
    """Chuẩn bị dữ liệu cho mô hình"""
```

```
    print("\n" + "="*60)
```

```
    print("CHUẨN BỊ DỮ LIỆU")
```

```
    print("="*60)
```

```
    # Chọn các features có thể dùng để dự đoán BMI
```

```
    # Loại bỏ BMI_calc vì nó là công thức tính từ Weight và Height
```

```
    feature_cols = [
```

```
        'Age', 'Weight (kg)', 'Height (m)', 'Max_BPM', 'Avg_BPM',
```

```
        'Resting_BPM', 'Session_Duration (hours)', 'Calories_Burned',
```

```
        'Fat_Percentage', 'Water_Intake (liters)', 'Workout_Frequency (days/week)',
```

```
        'Experience_Level', 'Carbs', 'Proteins', 'Fats', 'Calories'
```

```
    ]
```

```
    # Lọc các cột có trong dataframe
```

```
    available_features = [col for col in feature_cols if col in df.columns]
```

```
    # Tạo X và y
```

```
    X = df[available_features].copy()
```

```
    y = df["BMI"].copy()
```

```
    # Xử lý dữ liệu thiếu
```

```
    X = X.fillna(X.mean())
```

```

y = y.fillna(y.mean())

# Loại bỏ các dòng có giá trị inf hoặc NaN
mask = np.isfinite(X).all(axis=1) & np.isfinite(y)
X = X[mask]
y = y[mask]

print(f"\nSố features: {len(X.columns)}")
print(f"Số mẫu: {len(X)}")
print(f"\nFeatures được sử dụng:")
for i, col in enumerate(X.columns, 1):
    print(f" {i}. {col}")

return X, y

def train_model(X, y):
    """Huấn luyện mô hình hồi quy tuyến tính"""
    print("\n" + "="*60)
    print("HUẤN LUYỆN MÔ HÌNH")
    print("="*60)

    # Chia dữ liệu
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    print(f"\nDữ liệu training: {len(X_train)} mẫu")
    print(f"Dữ liệu test: {len(X_test)} mẫu")

    # Tạo và huấn luyện mô hình
    model = LinearRegression()
    model.fit(X_train, y_train)

    # Dự đoán
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

```

```
# Đánh giá mô hình
train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)
train_rmse = np.sqrt(train_mse)
test_rmse = np.sqrt(test_mse)
train_mae = mean_absolute_error(y_train, y_train_pred)
test_mae = mean_absolute_error(y_test, y_test_pred)
train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)
```

```
print("\n" + "-"*60)
print("KẾT QUẢ ĐÁNH GIÁ MÔ HÌNH")
print("-"*60)
print(f"\nTraining Set:")
print(f" R² Score: {train_r2:.4f}")
print(f" RMSE: {train_rmse:.4f}")
print(f" MAE: {train_mae:.4f}")
```

```
print(f"\nTest Set:")
print(f" R² Score: {test_r2:.4f}")
print(f" RMSE: {test_rmse:.4f}")
print(f" MAE: {test_mae:.4f}")
```

```
# Hiển thị hệ số
print("\n" + "-"*60)
print("HỆ SỐ CỦA MÔ HÌNH (Top 10)")
print("-"*60)
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
}).sort_values('Coefficient', key=abs, ascending=False)
```

```
print(coef_df.head(10).to_string(index=False))
```

```
return model, {
    'train_r2': train_r2,
    'test_r2': test_r2,
```

```

        'train_rmse': train_rmse,
        'test_rmse': test_rmse,
        'train_mae': train_mae,
        'test_mae': test_mae,
        'coefficients': coef_df.to_dict('records')
    }

```

```

def save_model(model, model_path='models/bmi_model.pkl'):

```

```

    """Lưu mô hình"""

```

```

    # Tạo thư mục nếu chưa có

```

```

    os.makedirs(os.path.dirname(model_path), exist_ok=True)

```

```

    with open(model_path, 'wb') as f:

```

```

        pickle.dump(model, f)

```

```

    print(f"\n✓ Đã lưu mô hình: {model_path}")

```

```

def load_model(model_path='models/bmi_model.pkl'):

```

```

    """Load mô hình"""

```

```

    with open(model_path, 'rb') as f:

```

```

        model = pickle.load(f)

```

```

    return model

```

```

def predict_bmi(model, features):

```

```

    """Dự đoán BMI từ các features"""

```

```

    return model.predict(features)

```

```

def main():

```

```

    """Hàm chính"""

```

```

    print("="*60)

```

```

    print("XÂY DỰNG MÔ HÌNH HỒI QUY TUYẾN TÍNH - DỰ ĐOÁN BMI")

```

```

    print("="*60)

```

```

    # Load dữ liệu

```

```

    print("\nĐang tải dữ liệu...")

```

```

    df = pd.read_csv('data/Final_data.csv')

```

```

    print(f"✓ Đã tải {len(df)} dòng dữ liệu")

```

```

# Chuẩn bị dữ liệu
X, y = prepare_data(df)

# Huấn luyện mô hình
model, metrics = train_model(X, y)

# Lưu mô hình
save_model(model)

print("\n" + "="*60)
print("✓ Hoàn thành!")
print("="*60)

return model, metrics

if __name__ == "__main__":
    main()

```

Dự đoán BMI

Nhập thông tin để dự đoán BMI bằng mô hình tuyến tính

[Biểu đồ](#) [Dự đoán](#)

Thông tin đầu vào

Tuổi (Age)	Cân nặng (kg)	Chiều cao (m)	Max BPM
Avg BPM	Resting BPM	Calories Burned	Fat Percentage
Lượng nước uống mỗi ngày (liters)	Tần suất tập luyện (ngày/tuần)	Thời lượng mỗi buổi (giờ)	Cấp độ kinh nghiệm

5. Sử dụng flask để trực quan hóa Dashboard và mô hình học máy đã sử dụng lên web

```

"""
Ứng dụng Flask để trực quan hóa Dashboard và mô hình học máy
"""

from flask import Flask, render_template, request, jsonify
import pandas as pd
import numpy as np

```

```

import os
from ml_model import prepare_data, load_model

app = Flask(__name__)
app.config['SECRET_KEY'] = 'your-secret-key-here'

# Load dữ liệu và mô hình (nhẹ, tránh log/thao tác dư thừa)
data_path = 'data/Final_data_cleaned.csv' if os.path.exists('data/Final_data_cleaned.csv') else 'data/Final_data.csv'
df = pd.read_csv(data_path)

# Chuẩn bị dữ liệu cho mô hình
X, y = prepare_data(df)
feature_columns = list(X.columns)

# Load mô hình; không tự huấn luyện để tránh chặn khởi động server
model_path = 'models/bmi_model.pkl'
model = load_model(model_path) if os.path.exists(model_path) else None

# Tự động huấn luyện nếu chưa có mô hình
if model is None:
    try:
        from ml_model import train_model, save_model
        model, _metrics = train_model(X, y)
        save_model(model, model_path)
    except Exception as e:
        # Cho phép server vẫn chạy, nhưng các API phụ thuộc model sẽ báo 503
        model = None

```